Ingeniería de Software II

Gestión de cambios

Universidad Autónoma de Entre Ríos
Facultad de Ciencia y Tecnología

Lic. Paolo Orundés Cardinali
orundescardinali.paolo@uader.edu.ar

Lic. Damián Cian
cian.damian@uader.edu.ar

Lic. Pamela Bonadeo
bonadeo.pamela@uader.edu.ar

Temas a desarrollar

Historia, Conceptos y Herramientas de la Gestión y Control de Cambios

1. **Introducción a la Gestión de cambios en software**
   - Que es la gestión de cambios, gestión de la configuración, control de cambios y control de versiones en software
   - Importancia del control de versiones en proyectos de software.
   - Problemas comunes antes o el no uso del control de versiones: pérdida de código, conflictos, versiones desorganizadas.
2. **Historia del Control de Versiones**
   - Primeros sistemas de control de cambios.
   - Evolución hacia los sistemas distribuidos (CVS, SVN, Git).
3. **Clasificación de Sistemas de Control de Versiones**
   - **Sistemas centralizados**: Características, ejemplos (CVS, SVN).
   - **Sistemas distribuidos**: Características, ejemplos (Git, Mercurial).
4. **Beneficios del Control de Versiones**
   - Colaboración y trabajo en equipo.
   - Seguimiento histórico de versiones.
   - Facilidades para revertir cambios.
5. **Herramientas Actuales de Control de Versiones**
   - Comparación de herramientas más usadas: Git, Mercurial, SVN.
   - Repositorios alojados en la nube: GitHub, GitLab, Bitbucket.

Práctica con Git, SSH y Repositorios

1. **Introducción a Git**
   - Conceptos fundamentales: commits, branches, merges, pull requests.
   - Flujo de trabajo básico de Git: staging area, commits, y branches.
2. **Generación y Uso de Claves SSH**
   - Explicación de claves SSH.
   - Pasos para generar claves SSH.
   - Configuración de claves SSH en plataformas como GitHub y GitLab.
3. **Repositorios Remotos y Comandos Básicos de Git**
   - Creación y clonación de repositorios.
   - Comandos esenciales: git init, git clone, git add, git commit, git push, git pull.
   - Manejo de ramas: creación y fusión (git branch, git merge).
4. **Resolución de Conflictos en Git**
   - Cómo surgen los conflictos.
   - Estrategias para resolverlos de manera eficiente.
5. **Buenas Prácticas en la Gestión de Cambios**
   - Nombres claros en los commits.
   - Hacer commits pequeños y frecuentes.
   - Uso correcto de ramas para diferentes flujos de trabajo (Git Flow).
6. **Integración Continua y Automatización**
   - Breve introducción a herramientas de integración continua (Jenkins, GitHub Actions).
   - Automatización de pruebas y despliegue.

Introducción a la gestión de cambios en Software

Antes de comenzar con control de cambios debemos repasar los conceptos básicos de: gestión de cambios, gestión de la configuración, control de cambios y control de versiones.

1. Gestión de Cambios en Software

- **Definición**: Es el proceso de identificar, documentar, evaluar, y controlar los cambios propuestos en un proyecto de software. Incluye la evaluación de cómo estos cambios afectarán el proyecto en términos de tiempo, recursos y riesgos.
- **Objetivo**: Asegurar que cualquier cambio en los requisitos, diseño, o implementación de un proyecto de software sea gestionado de manera eficiente, minimizando riesgos y asegurando que no se comprometan los objetivos del proyecto.
- **Ejemplo**: Un cliente solicita añadir una nueva funcionalidad a una aplicación existente. La gestión de cambios evalúa el impacto de este cambio y asegura que se implemente de manera controlada.

2. Gestión de la Configuración de Software (SCM - Software Configuration Management)

- **Definición**: Es el proceso de gestionar de manera sistemática todos los artefactos de un proyecto de software, como el código fuente, la documentación, los entornos de desarrollo y los componentes necesarios para la construcción de un producto. SCM asegura que todas las versiones y componentes estén bajo control y disponibles para el equipo.
- **Objetivo**: Proporcionar control y trazabilidad sobre los elementos del sistema a lo largo del ciclo de vida del proyecto, asegurando que cada cambio se realice de forma estructurada y controlada.
- **Ejemplo**: Un equipo que desarrolla un sistema de software mantiene una base de datos central con todas las versiones del código, documentación, archivos de configuración, y scripts de despliegue.

3. Control de Cambios

- **Definición**: Es un subconjunto dentro de la **gestión de la configuración** que se enfoca específicamente en cómo se aprueban, implementan y rastrean los cambios realizados en el software. El control de cambios establece un proceso formal para solicitar, evaluar, aprobar o rechazar cambios.
- **Objetivo**: Garantizar que los cambios sean aprobados adecuadamente y no afecten negativamente el software ni los objetivos del proyecto.
- **Ejemplo**: Una solicitud de cambio para corregir un bug crítico en producción debe pasar por el proceso de control de cambios para asegurar que no se introduzcan nuevos errores y que todos los interesados estén informados.

4. Control de Versiones

- **Definición**: Es el proceso de gestionar múltiples versiones o variantes de un software a lo largo del tiempo. El control de versiones asegura que los desarrolladores puedan trabajar de manera colaborativa en diferentes características o correcciones, y que se pueda volver a versiones anteriores si es necesario.
- **Objetivo**: Proveer un historial de todos los cambios realizados en el código o en otros componentes, permitiendo la colaboración, la recuperación de versiones anteriores y la comparación de cambios.
- **Ejemplo**: Usar **Git** para gestionar las versiones del código fuente de una aplicación, donde cada desarrollador puede crear ramas, hacer commits y fusionar los cambios de manera controlada.

Diferencias Clave:

- **Gestión de Cambios**: Enfocada en los cambios a nivel de requisitos, diseño o funcionalidades. Se evalúa el impacto del cambio en todo el proyecto.
- **Gestión de la Configuración**: Se ocupa de la organización y control de todos los componentes del software a lo largo de su ciclo de vida (código, documentación, etc.).
- **Control de Cambios**: Un proceso formal dentro de la gestión de configuración que asegura que los cambios aprobados se implementen correctamente.
- **Control de Versiones**: Proceso técnico que permite gestionar diferentes versiones del software, facilitando la colaboración y la trazabilidad de los cambios en el código fuente.

Estos conceptos se complementan para asegurar un desarrollo de software ordenado, seguro y eficiente.

Dados estos conceptos vamos a enfocarnos en este último: Control de versiones

Profundizando el conocimiento sobre control de versiones

El control de versiones, también conocido como "control de código fuente", es la práctica de rastrear y gestionar los cambios en el código de software. Los sistemas de control de versiones son herramientas de software que ayudan a los equipos de software a gestionar los cambios en el código fuente a lo largo del tiempo. A medida que los entornos de desarrollo se aceleran, los sistemas de control de versiones ayudan a los equipos de software a trabajar de forma más rápida e inteligente. Son especialmente útiles para los equipos de DevOps, ya que les ayudan a reducir el tiempo de desarrollo y a aumentar las implementaciones exitosas.

El software de control de versiones realiza un seguimiento de todas las modificaciones en el código en un tipo especial de base de datos. Si se comete un error, los desarrolladores pueden ir hacia atrás en el tiempo y comparar las versiones anteriores del código para ayudar a resolver el error, al tiempo que se minimizan las interrupciones para todos los miembros del equipo.

Para casi todos los proyectos de software, el código fuente es como las joyas de la corona, un activo valioso cuyo valor debe protegerse. Para la mayoría de equipos de software, el código fuente es un repositorio del conocimiento de valor incalculable y de la comprensión sobre el dominio del problema que los desarrolladores han recopilado y perfeccionado con un esfuerzo cuidadoso. El control de versiones protege el código fuente tanto de las catástrofes como del deterioro casual de los errores humanos y las consecuencias accidentales.

Los desarrolladores de software que trabajan en equipos están escribiendo continuamente nuevo código fuente y cambiando el que ya existe. El código de un proyecto, una aplicación o un componente de software normalmente se organiza en una estructura de carpetas o "árbol de archivos". Un desarrollador del equipo podría estar trabajando en una nueva función mientras otro desarrollador soluciona un error no relacionado cambiando código. Cada desarrollador podría hacer sus cambios en varias partes del árbol de archivos.

El control de versiones ayuda a los equipos a resolver este tipo de problemas al realizar un seguimiento de todos los cambios individuales de cada colaborador y al contribuir a evitar que el trabajo concurrente entre en conflicto. Los cambios realizados en una parte del software pueden ser incompatibles con los que ha hecho otro desarrollador que está trabajando al mismo tiempo. Este problema debería detectarse y solucionarse de manera ordenada sin bloquear el trabajo del resto del equipo. Además, en

todo el desarrollo de software, cualquier cambio puede introducir nuevos errores por sí mismo y el nuevo software no es fiable hasta que se prueba. De este modo, las pruebas y el desarrollo van de la mano hasta que está lista una nueva versión.

Un buen software de control de versiones soporta el flujo de trabajo preferido de un desarrollador sin imponer una forma determinada de trabajar. Idealmente, también funciona en cualquier plataforma, en vez de ordenar qué sistema operativo o cadena de herramientas deben utilizar los desarrolladores. Los sistemas de control de versiones excepcionales facilitan un flujo sencillo y continúo de cambios en el código en vez del mecanismo frustrante y burdo del bloqueo de archivos, que da luz verde a un desarrollador a expensas de bloquear el progreso de los demás.

Los equipos de software que no utilizan ninguna forma de control de versiones a menudo se encuentran con problemas como no saber qué cambios que se han hecho están disponibles para los usuarios o la creación de cambios incompatibles entre dos partes no relacionadas que tienen que desvincularse y revisarse exhaustivamente. Si eres un desarrollador que nunca ha utilizado el control de versiones, puede que hayas añadido versiones a tus archivos, quizás con sufijos como "final" o "más reciente", y que después hayas tenido que enfrentarte con una nueva versión final. Quizás has convertido en comentarios bloques de código, porque quieres desactivar una determinada función sin eliminar el código, con el miedo de que pueda utilizarse más adelante. El control de versiones es una forma de solucionar estos problemas.

El software de control de versiones es una parte esencial del día a día de las prácticas profesionales del equipo de software moderno. Los desarrolladores de software individuales que están acostumbrados a trabajar con un sistema de control de versiones potente en sus equipos suelen reconocer el increíble valor que el control de versiones también les da incluso en los proyectos pequeños en los que trabajan solos. Una vez acostumbrados a las potentes ventajas de los sistemas de control de versiones, muchos desarrolladores no se plantearían trabajar sin ellos incluso para los proyectos que no son de software.

Importancia del control de versiones en proyectos de software.

Utilizar un software de control de versiones es una práctica recomendada para los equipos de software y de DevOps de alto rendimiento. El control de versiones también ayuda a los desarrolladores a moverse más rápido y permite que los equipos de software mantengan la eficiencia y la agilidad a medida que el equipo se escala para incluir más desarrolladores.

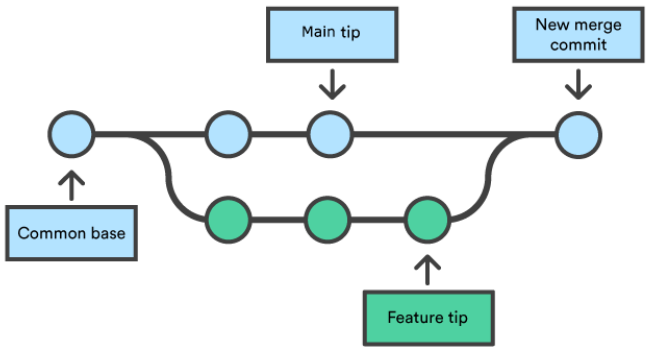
Los sistemas de control de versiones (VCS) han experimentado grandes mejoras en las últimas décadas y algunos son mejores que otros. Los VCS a veces se conocen como herramientas de SCM (Gestión del código fuente) o RCS (Sistema de control de revisiones). Una de las herramientas de VCS más populares hoy en día se llama Git. Git es un VCS distribuido, una categoría conocida como DVCS, que explicaremos con más detalle después. Al igual que muchos de los sistemas de VCS más populares disponibles hoy en día, Git es gratuito y de código abierto. Independientemente del nombre que tengan, o del sistema que se utilice, las principales ventajas que deberías esperar del control de versiones son las siguientes.

1. **Un completo historial de cambios a largo plazo de todos los archivos**. Esto quiere decir todos los cambios realizados por muchas personas a lo largo de los años. Los cambios incluyen la creación y la eliminación de los archivos, así como los cambios de sus contenidos. Las diferentes herramientas de VCS difieren en lo bien que gestionan el cambio de nombre y el movimiento de los archivos. Este historial también debería incluir el autor, la fecha y notas escritas sobre el propósito de cada cambio. Tener el historial completo permite volver a las versiones anteriores para ayudar a analizar la causa raíz de los errores y es crucial cuando se tiene que solucionar problemas en las versiones anteriores del software.

Si se está trabajando de forma activa en el software, casi todo puede considerarse una "versión anterior" del software.



2. **Creación de ramas y fusiones**. Si se tiene a miembros del equipo trabajando al mismo tiempo, es algo evidente; pero incluso las personas que trabajan solas pueden beneficiarse de la capacidad de trabajar en flujos independientes de cambios. La creación de una "rama" en las herramientas de VCS mantiene múltiples flujos de trabajo independientes los unos de los otros al tiempo que ofrece la facilidad de volver a fusionar ese trabajo, lo que permite que los desarrolladores verifiquen que los cambios de cada rama no entran en conflicto. Muchos equipos de software adoptan la práctica de crear ramas para cada función o quizás para cada publicación, o ambas. Existen muchos flujos de trabajo diferentes que los equipos pueden elegir cuando deciden cómo utilizar la creación de las ramas y las fusiones en VCS.



3. **Trazabilidad**. Ser capaz de trazar cada cambio que se hace en el software y conectarlo con un software de gestión de proyectos y seguimiento de errores como Jira, además de ser capaz de anotar cada cambio con un mensaje que describa el propósito y el objetivo del cambio, no solo te ayuda con el análisis de la causa raíz y la recopilación de información. Tener el historial anotado del código a tu alcance cuando estás leyendo el código, intentando entender lo que hace y por qué se ha diseñado así, puede permitir a los desarrolladores hacer cambios correctos y que estén en línea con el diseño previsto a largo plazo del sistema. Esto puede ser especialmente importante para trabajar de manera eficaz con código heredado y es esencial para que los desarrolladores puedan calcular el trabajo futuro con precisión.

Aunque se puede desarrollar software sin utilizar ningún control de versiones, hacerlo somete al proyecto a un gran riesgo que ningún equipo profesional debería aceptar. Así que la pregunta no es si utilizar el control de versiones, sino qué sistema de control de versiones usar.

Hay muchas opciones, pero aquí vamos a centrarnos solo en una, Git.

Problemas comunes antes y del no uso del control de versiones

1. Pérdida de Código

- **Descripción**: Sin control de versiones, es fácil sobrescribir o eliminar accidentalmente el código. Si varios desarrolladores trabajan sobre los mismos archivos, existe un alto riesgo de perder trabajo debido a conflictos o sobrescritura de archivos.

- **Ejemplo**: Un desarrollador accidentalmente guarda una versión anterior del código en lugar de una actualizada, eliminando cambios recientes.

2. Dificultad para Colaborar en Equipo

- **Descripción**: En un equipo sin control de versiones, coordinar los cambios es complejo. Cada desarrollador necesita acceder a la misma versión de código y cualquier cambio debe compartirse manualmente, lo cual es lento y propenso a errores.

- **Ejemplo**: Si dos desarrolladores hacen cambios en el mismo archivo, no hay una manera automática de fusionar esos cambios y uno de ellos podría sobrescribir el trabajo del otro.

3. Falta de Historial de Cambios

- **Descripción**: Sin un sistema de control de versiones, no hay registro detallado de los cambios realizados en el código, lo que dificulta la revisión del progreso y el diagnóstico de problemas.

- **Ejemplo**: Si surge un error nuevo, el equipo no puede retroceder a una versión anterior ni identificar fácilmente cuándo se introdujo el cambio que lo causó.

4. Imposibilidad de Revertir Cambios

- **Descripción**: Cuando ocurre un error crítico, sin control de versiones es difícil (si no imposible) volver a una versión funcional anterior. Esto puede llevar a una pérdida significativa de tiempo y recursos.

- **Ejemplo**: Un cambio en el código rompe la aplicación en producción, pero al no haber una copia de respaldo, los desarrolladores deben reconstruir manualmente una versión estable.

5. Problemas con el Lanzamiento de Versiones

- **Descripción**: Sin un sistema de control de versiones, es difícil gestionar las diferentes versiones del software (por ejemplo, una versión en producción y una en desarrollo). Esto puede llevar a confusión y a errores al desplegar o probar el software.

- **Ejemplo**: Un equipo despliega una actualización en producción que contiene cambios de desarrollo no finalizados porque no hay una separación clara entre las versiones.

6. Complicaciones en la Resolución de Conflictos

- **Descripción**: Cuando varios desarrolladores editan el mismo archivo, sin un sistema de control de versiones no hay forma de gestionar y resolver conflictos automáticamente. Esto obliga a los desarrolladores a coordinar manualmente cada cambio, lo cual es ineficiente y propenso a errores.

- **Ejemplo**: Dos desarrolladores editan la misma función en un archivo. Sin control de versiones, deben revisar manualmente cada cambio y ajustar el código para evitar errores.

7. Ausencia de Trazabilidad y Auditoría

- **Descripción**: La trazabilidad se refiere a la capacidad de saber quién hizo qué cambio y cuándo, algo que el control de versiones facilita enormemente. Sin esto, no se puede realizar una auditoría detallada de las modificaciones en el código.

- **Ejemplo**: Un bug crítico se introduce en el código, pero al no poder identificar al autor ni el momento exacto en que se hizo el cambio, el proceso de depuración se vuelve largo y complejo.

8. Falta de Seguridad en el Código

- **Descripción**: Sin un sistema de control de versiones, el código suele estar disperso en varias computadoras, lo que aumenta el riesgo de pérdida de datos por fallos de hardware o errores humanos.

- **Ejemplo**: Un miembro del equipo borra accidentalmente la última versión del código en su computadora, y como no hay un repositorio centralizado, todos pierden acceso a la versión más reciente del software.

9. Dificultad para Implementar Entornos de Pruebas

- **Descripción**: Los sistemas de control de versiones permiten manejar ramas y versiones del software, lo que facilita crear entornos de pruebas y trabajar en varias características o correcciones de manera aislada. Sin esta posibilidad, probar cambios antes de integrarlos es mucho más complicado.

- **Ejemplo**: Un desarrollador necesita probar una nueva característica, pero como no hay control de versiones, debe realizar pruebas directamente en el código en producción, aumentando el riesgo de errores.

2. Historia del Control de Versiones

Primeros sistemas de control de cambios

- Surgieron en los años 70 para gestionar versiones de documentos y software en equipos grandes.
- Source Code Control System (SCCS): Fue uno de los primeros sistemas de control de versiones, desarrollado por Bell Labs en 1972.
- Revision Control System (RCS): Desarrollado en los años 80, fue ampliamente utilizado y sentó las bases para los sistemas más modernos.

Evolución hacia sistemas distribuidos

- **Concurrent Versions System (CVS)**: Años 80. Fue el primer sistema que permitía trabajo en equipo, pero centralizado. Permitía que varios usuarios trabajaran en el mismo proyecto, pero los cambios se almacenaban en un único servidor.
- **Subversion (SVN)**: Creado en 2000 para mejorar CVS. Aportó mejoras como la capacidad de realizar commits atómicos, pero seguía siendo centralizado.
- **Git**: Creado en 2005 por Linus Torvalds, introdujo un modelo distribuido, permitiendo que cada usuario tuviera una copia completa del proyecto. Esto mejoró la seguridad, el rendimiento y la colaboración.

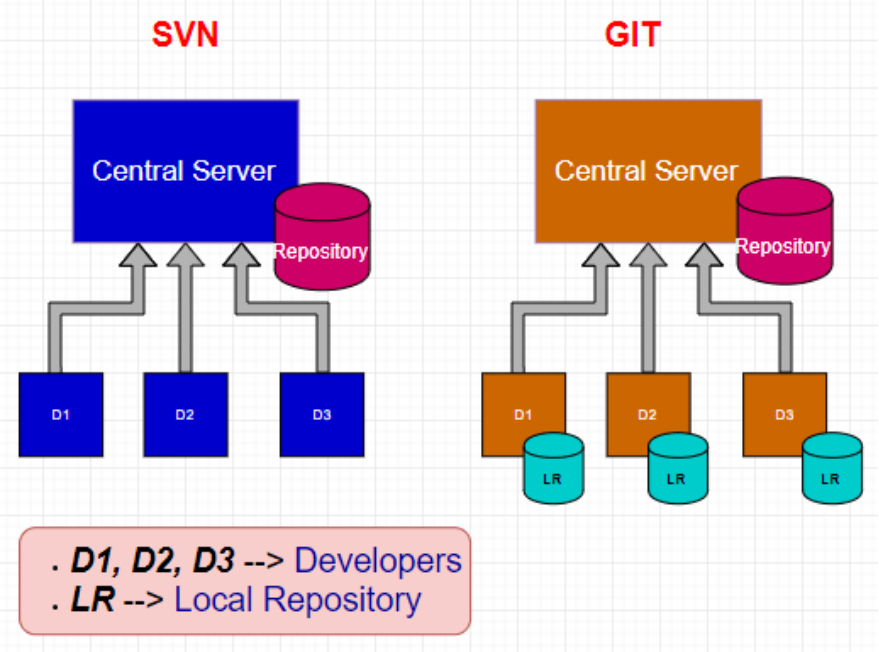
Clasificación de Sistemas de Control de Versiones

Sistemas Centralizados

- **Características**: Tienen un único servidor que almacena el código. Los desarrolladores se conectan al servidor para obtener y actualizar el código. Es necesario tener conexión constante al servidor para trabajar.
- **Ejemplos**:
  - **CVS**: Usado en los años 80 y 90, es uno de los primeros en implementar el control de versiones.
  - **SVN**: Permite commits atómicos, control de acceso y gestión de cambios, pero sigue siendo centralizado.
- **Ventajas**: Fácil de administrar, ya que hay un solo repositorio central.
- **Desventajas**: La dependencia del servidor central hace que el sistema sea vulnerable a fallas y limita el trabajo sin conexión.

Sistemas Distribuidos

- **Características**: Cada desarrollador tiene una copia completa del repositorio, incluyendo el historial completo de cambios. Esto permite trabajar de manera descentralizada y hacer cambios sin conexión.
- **Ejemplos**:
  - **Git**: Almacena todo el historial de versiones en cada clon local, permitiendo un trabajo rápido y seguro.
  - **Mercurial**: Similar a Git en su enfoque distribuido, pero con una curva de aprendizaje algo más accesible.
- **Ventajas**: Mayor seguridad (copia completa del proyecto en cada máquina), trabajo sin conexión, colaboración descentralizada.
- **Desventajas**: Necesita mayor espacio en disco, ya que cada usuario tiene una copia completa.



Beneficios del Control de Versiones

Colaboración y Trabajo en Equipo

- Facilita que múltiples desarrolladores trabajen en paralelo sin interferir entre sí. Los cambios se pueden integrar de manera controlada.
- Las ramas permiten trabajar en nuevas características de forma aislada y fusionarlas cuando estén listas.

Seguimiento Histórico de Versiones

- Proporciona un historial de cada cambio, incluyendo el autor, la fecha y el motivo. Esto facilita la auditoría y permite entender cómo ha evolucionado el proyecto.
- Ayuda a localizar la introducción de errores y deshacer cambios si es necesario.

Facilidades para Revertir Cambios

- Permite volver a versiones anteriores si se introduce un error o si un cambio causa problemas. Esto evita la pérdida de tiempo y minimiza el riesgo de errores críticos en producción.
- Se pueden crear “puntos de control” en versiones estables que facilitan las actualizaciones y el mantenimiento del software.

Herramientas Actuales de Control de Versiones

Comparación de Herramientas Más Usadas

- **Git:**
  - Ventajas: Sistema distribuido con alto rendimiento, popularidad y soporte en la mayoría de servicios de repositorios (GitHub, GitLab).
  - Desventajas: Curva de aprendizaje inicial, especialmente para principiantes.
- **Mercurial:**
  - Ventajas: Similar a Git en funcionalidad distribuida, pero con comandos más sencillos y menos errores en la gestión de repositorios.

    - Desventajas: Menos popular que Git, por lo que tiene menor soporte en herramientas de repositorio.
- **SVN:**
    - Ventajas: Aún popular en empresas que prefieren un sistema centralizado y más sencillo.
    - Desventajas: Menos flexible y eficiente que los sistemas distribuidos. Requiere conexión al servidor central.

Repositorios Alojados en la Nube

- **GitHub**: Popular por su enfoque social, colaboración en proyectos de código abierto, y funcionalidades como el seguimiento de issues y los pull requests.
- **GitLab**: Ofrece un completo sistema de integración y despliegue continuo (CI/CD) y tiene una versión de código abierto.
- **Bitbucket**: Integración con herramientas de Atlassian como Jira. Soporte para Git y Mercurial, útil para proyectos privados en empresas.

Ventajas de Repositorios en la Nube:

- Acceso remoto y fácil a los proyectos.
- Funcionalidades de colaboración, CI/CD, y herramientas de gestión de proyectos.
- Mayor seguridad en la copia de los datos y protección ante pérdidas de datos.

GIT

https://git-scm.com/

¿Qué es GIT?

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

Git es un sistema de control de versiones distribuido, diseñado para rastrear y gestionar cambios en el código fuente de proyectos de software. Fue creado por Linus Torvalds en 2005 para mejorar el desarrollo colaborativo, permitiendo que múltiples personas trabajen en el mismo proyecto sin sobrescribir el trabajo de los demás.

Puntos clave de Git:

- **Control de versiones**: Permite almacenar diferentes versiones del proyecto, facilitando la revisión y restauración de versiones anteriores.
- **Distribuido**: Cada copia del proyecto es un repositorio completo, lo cual permite trabajar sin conexión y hacer "commits" localmente.
- **Trabajo en equipo**: Facilita la colaboración mediante ramas (*branches*) donde cada colaborador puede trabajar sin afectar directamente la versión principal, integrando los cambios al final.
- **Historial detallado**: Guarda un registro de cada cambio, permitiendo ver qué se cambió, cuándo y por quién.
- **Popularidad y soporte**: Es el sistema de control de versiones más utilizado en la actualidad, con plataformas como GitHub y GitLab que lo potencian para el desarrollo colaborativo y la gestión de proyectos.

Git se usa en cualquier flujo de trabajo de desarrollo, ya que garantiza un control eficiente sobre el código y la colaboración en equipo, especialmente en proyectos de larga duración.

Inicio - Sobre el Control de Versiones - Fundamentos de Git

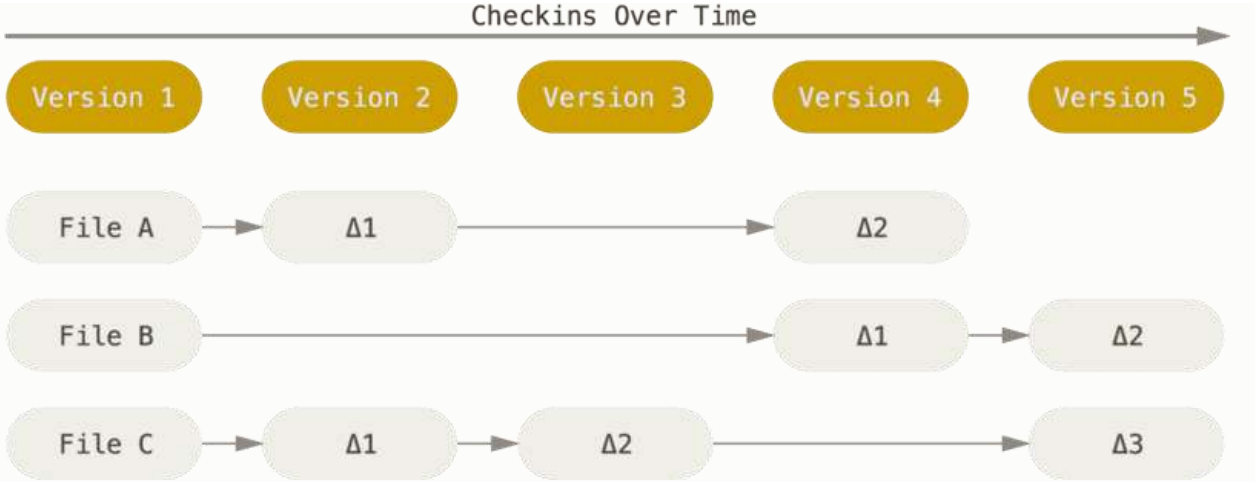
https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Fundamentos-de-Git

Entonces, ¿qué es Git en pocas palabras? Es muy importante entender bien esta sección, porque si entiendes lo que es Git y los fundamentos de cómo funciona, probablemente te será mucho más fácil usar Git efectivamente. A medida que aprendas Git, intenta olvidar todo lo que posiblemente conoces acerca de otros VCS como Subversion y Perforce. Hacer esto te ayudará a evitar confusiones sutiles a la hora de utilizar la herramienta. Git almacena y maneja la información de forma muy diferente a esos otros sistemas, a pesar de que su interfaz de usuario es bastante similar. Comprender esas diferencias evitará que te confundas a la hora de usarlo.

Copias instantáneas, no diferencias

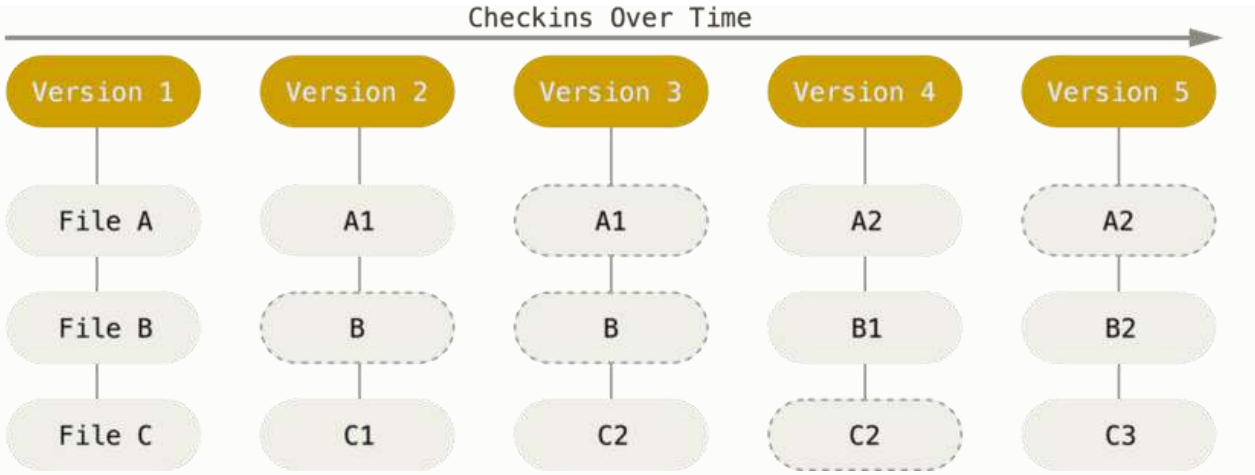
La principal diferencia entre Git y cualquier otro VCS (incluyendo Subversion y sus amigos) es la forma en la que manejan sus datos. Conceptualmente, la mayoría de los otros sistemas almacenan la información como una lista de cambios en los archivos. Estos sistemas (CVS, Subversion, Perforce,

Bazaar, etc.) manejan la información que almacenan como un conjunto de archivos y las modificaciones hechas a cada uno de ellos a través del tiempo.



Almacenamiento de datos como cambios en una versión de la base de cada archivo.

Git no maneja ni almacena sus datos de esta forma. Git maneja sus datos como un conjunto de copias instantáneas de un sistema de archivos miniatura. Cada vez que confirmas un cambio, o guardas el estado de tu proyecto en Git, él básicamente toma una foto del aspecto de todos tus archivos en ese momento y guarda una referencia a esa copia instantánea. Para ser eficiente, si los archivos no se han modificado Git no almacena el archivo de nuevo, sino un enlace al archivo anterior idéntico que ya tiene almacenado. Git maneja sus datos como una secuencia de copias instantáneas.



Almacenamiento de datos como instantáneas del proyecto a través del tiempo.

Esta es una diferencia importante entre Git y prácticamente todos los demás VCS. Hace que Git reconsidere casi todos los aspectos del control de versiones que muchos de los demás sistemas copiaron de la generación anterior. Esto hace que Git se parezca más a un sistema de archivos miniatura con algunas herramientas tremendamente poderosas desarrolladas sobre él, que a un VCS. Exploraremos algunos de los beneficios que obtienes al modelar tus datos de esta manera cuando veamos ramificación (branching) en Git en el (véase [ch03-git-branching]).

Casi todas las operaciones son locales

La mayoría de las operaciones en Git sólo necesitan archivos y recursos locales para funcionar. Por lo general no se necesita información de ningún otro computador de tu red. Si estás acostumbrado a un CVCS donde la mayoría de las operaciones tienen el costo adicional del retardo de la red, este aspecto

de Git te va a hacer pensar que los dioses de la velocidad han bendecido Git con poderes sobrenaturales. Debido a que tienes toda la historia del proyecto ahí mismo, en tu disco local, la mayoría de las operaciones parecen prácticamente inmediatas.

Por ejemplo, para navegar por la historia del proyecto, Git no necesita conectarse al servidor para obtener la historia y mostrártela - simplemente la lee directamente de tu base de datos local. Esto significa que ves la historia del proyecto casi instantáneamente. Si quieres ver los cambios introducidos en un archivo entre la versión actual y la de hace un mes, Git puede buscar el archivo de hace un mes y hacer un cálculo de diferencias localmente, en lugar de tener que pedirle a un servidor remoto que lo haga, u obtener una versión antigua desde la red y hacerlo de manera local.

Esto también significa que hay muy poco que no puedes hacer si estás desconectado o sin VPN. Si te subes a un avión o a un tren y quieres trabajar un poco, puedes confirmar tus cambios felizmente hasta que consigas una conexión de red para subirlos. Si te vas a casa y no consigues que tu cliente VPN funcione correctamente, puedes seguir trabajando. En muchos otros sistemas, esto es imposible o muy engorroso. En Perforce, por ejemplo, no puedes hacer mucho cuando no estás conectado al servidor. En Subversion y CVS, puedes editar archivos, pero no puedes confirmar los cambios a tu base de datos (porque tu base de datos no tiene conexión). Esto puede no parecer gran cosa, pero te sorprendería la diferencia que puede suponer.

Git tiene integridad

Todo en Git es verificado mediante una suma de comprobación (checksum en inglés) antes de ser almacenado, y es identificado a partir de ese momento mediante dicha suma. Esto significa que es imposible cambiar los contenidos de cualquier archivo o directorio sin que Git lo sepa. Esta funcionalidad está integrada en Git al más bajo nivel y es parte integral de su filosofía. No puedes perder información durante su transmisión o sufrir corrupción de archivos sin que Git sea capaz de detectarlo.

El mecanismo que usa Git para generar esta suma de comprobación se conoce como hash SHA-1. Se trata de una cadena de 40 caracteres hexadecimales (0-9 y a-f), y se calcula con base en los contenidos del archivo o estructura del directorio en Git. Un hash SHA-1 se ve de la siguiente forma:

```
24b9da6552252987aa493b52f8696cd6d3b00373
```

Verás estos valores hash por todos lados en Git, porque son usados con mucha frecuencia. De hecho, Git guarda todo no por nombre de archivo, sino por el valor hash de sus contenidos.

Git generalmente solo añade información

Cuando realizas acciones en Git, casi todas ellas sólo añaden información a la base de datos de Git. Es muy difícil conseguir que el sistema haga algo que no se pueda enmendar, o que de algún modo borre información. Como en cualquier VCS, puedes perder o estropear cambios que no has confirmado todavía. Pero después de confirmar una copia instantánea en Git es muy difícil perderla, especialmente si envías tu base de datos a otro repositorio con regularidad.

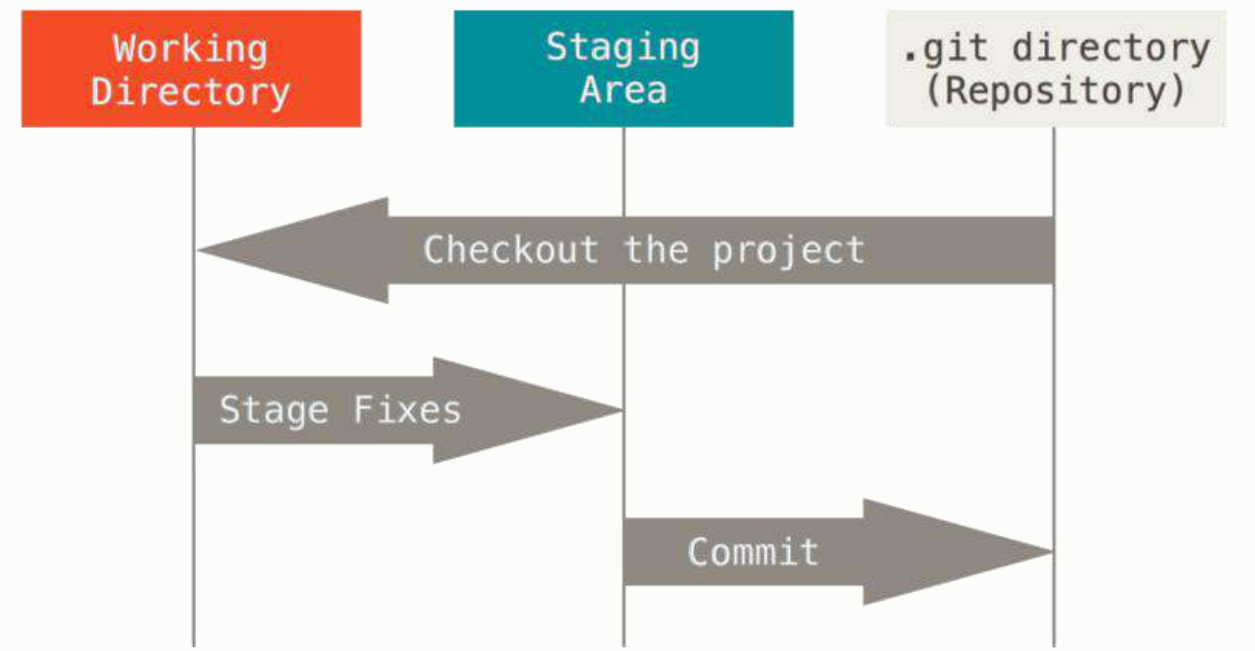
Esto hace que usar Git sea un placer, porque sabemos que podemos experimentar sin peligro de estropear gravemente las cosas. Para un análisis más exhaustivo de cómo almacena Git su información y cómo puedes recuperar datos aparentemente perdidos, ver Deshacer Cosas.

Los Tres Estados

Ahora presta atención. Esto es lo más importante que debes recordar acerca de Git si quieres que el resto de tu proceso de aprendizaje prosiga sin problemas. Git tiene tres estados principales en los que se pueden encontrar tus archivos: **confirmado (committed), modificado (modified), y preparado (staged)**.

- **Confirmado**: significa que los datos están almacenados de manera segura en tu base de datos local.
- **Modificado**: significa que has modificado el archivo, pero todavía no lo has confirmado a tu base de datos.
- **Preparado**: significa que has marcado un archivo modificado en su versión actual para que vaya en tu próxima confirmación.

Esto nos lleva a las tres secciones principales de un proyecto de Git: El directorio de Git (Git directory), el directorio de trabajo (working directory), y el área de preparación (staging area).



Directorio de trabajo, área de almacenamiento y el directorio Git.

El **directorio de Git** es donde se almacenan los metadatos y la base de datos de objetos para tu proyecto. Es la parte más importante de Git, y es lo que se copia cuando clonas un repositorio desde otra computadora.

El **directorio de trabajo** es una copia de una versión del proyecto. Estos archivos se sacan de la base de datos comprimida en el directorio de Git, y se colocan en disco para que los puedas usar o modificar.

El **área de preparación** es un archivo, generalmente contenido en tu directorio de Git, que almacena información acerca de lo que va a ir en tu próxima confirmación. A veces se le denomina índice ("index"), pero se está convirtiendo en estándar el referirse a ella como el área de preparación.

El flujo de trabajo básico en Git es algo así:

1. Modificas una serie de archivos en tu directorio de trabajo.
2. Preparas los archivos, añadiéndolos a tu área de preparación.
3. Confirmas los cambios, lo que toma los archivos tal y como están en el área de preparación y almacena esa copia instantánea de manera permanente en tu directorio de Git.

Si una versión concreta de un archivo está en el directorio de Git, se considera confirmada (committed). Si ha sufrido cambios desde que se obtuvo del repositorio, pero ha sido añadida al área de preparación, está preparada (staged). Y si ha sufrido cambios desde que se obtuvo del repositorio, pero no se ha preparado, está modificada (modified). En [ch02-git-basics] aprenderás más acerca de estos estados y de cómo puedes aprovecharlos o saltarte toda la parte de preparación.

Generación y Uso de Claves SSH

Las claves SSH son pares de claves (una pública y otra privada) que se utilizan para autenticarte de forma segura en servidores y plataformas de control de versiones, como GitHub, GitLab o Bitbucket. Usarlas evita que tengas que introducir tu usuario y contraseña cada vez que haces una acción en estos sistemas.

¿Qué es una clave SSH?

Clave pública: es una clave que puedes compartir con otros. Esta se sube al servidor o servicio en el que deseas autenticarte.

Clave privada: esta clave debe permanecer secreta y almacenarse de forma segura en tu computadora. Solo tú debes tener acceso a ella.

Cuando intentas conectar con el servidor, se utiliza la clave privada para autenticarte y permitirte el acceso.

Cómo Generar una Clave SSH

Windows

Para crear un par de claves en sistemas operativos Microsoft Windows:

1. Descargar PuTTy Key Generator PuTTygen.exe archivo y ejecutarlo.
2. En el menú **Clave**, seleccione **CLAVE RSA SSH-2**.
3. En **Parámetros**, seleccione la opción **RSA** .
4. En **El campo Número de bits en clave generada**, escriba al menos 2048, idealmente 4096.
5. En **Acciones**, seleccione **Generar**.
6. Aparece una barra de progreso. En el área en blanco de la barra, mueva el puntero en un patrón aleatorio para completar la generación del par de claves.
7. Cuando se complete la generación, copie el contenido de la **clave pública para pegarlo en el campo de archivo authorized_keys OpenSSH** y después en una aplicación de editor de texto (como el Bloc de notas).

Importante

Copie el ***texto completo*** de la clave pública, incluido `ssh-rsa`, en el campo Clave pública SSH al **configurar SFTP en Configuración general**.

10. Guarde el archivo como **vivaglint-aaaammdd.pub**.

11. **No** se requieren los campos De contraseña de contraseña y **Confirmar frase de contraseña** . Deje en blanco sin frase de contraseña.
12. Seleccione **Guardar clave privada** para guardar el archivo de clave privada. Esta clave le permite conectarse a SFTP en una aplicación FTP dedicada.

Macintosh o Linux

Para crear un par de claves en sistemas operativos Macintosh o Linux:

1. Abra una ventana **terminal**. En sistemas operativos Macintosh, busque **Terminalen dock o** en la carpeta **Utilidades**.
2. Escriba el siguiente comando: `ssh-keygen -b 2048 -t rsa -C "your_username" -f filename.`
    - La longitud, determinada por -b, debe ser al menos 2048, idealmente 4096.
    - Actualice **los valores de your_username** y **nombre de archivo** .
    - Ejemplo: `ssh-keygen -b 2048 -t rsa -C "jsmith" -f vivaglint-yyyymmdd.`
3. Seleccione **Entrar** y siga las indicaciones.
4. No se requiere una frase de contraseña. Proporcione una frase de contraseña o déjela en blanco y seleccione **Entrar** para continuar.
5. Cuando se genera correctamente, verá estos mensajes:
    - Su identificación se ha guardado en **nombre de archivo**.
    - La clave pública se ha guardado en **filename.pub**.
6. Busque los archivos de clave pública y privada en **Finder**.
    - Los dos archivos de clave se denominan en función de lo que se especificó en el comando Terminal.
        1. Ejemplo: **vivaglint-aaaammdd** y **vivaglint-aaaammdd.pub**
        2. Sin ningún nombre de archivo especificado, los archivos clave aparecen como: **id_rsa** y **id_rsa.pub**.
7. Abra el archivo .pub (la clave pública SSH) en un editor de texto (como TextEdit) y copie el contenido.

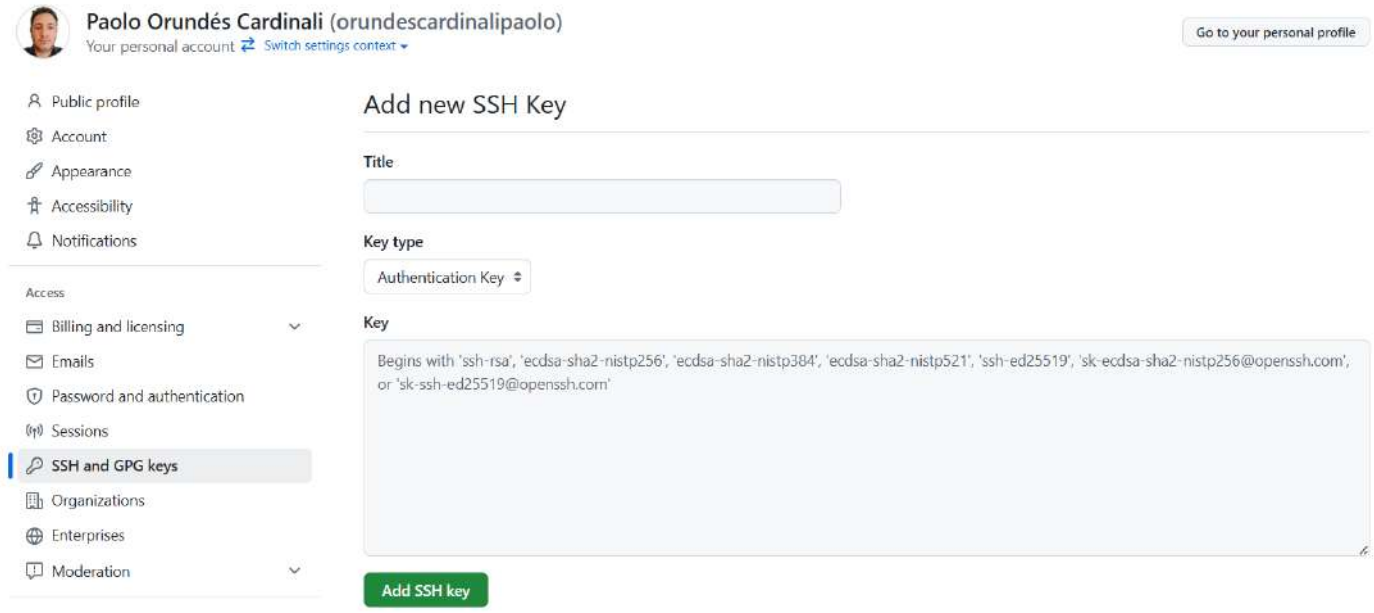
Importante

Copie el ***texto completo*** de la clave pública, incluido `ssh-rsa`, en el campo Clave pública SSH al **configurar SFTP en Configuración general**.

Registrar Clave SSH en GitHub o Gitlab

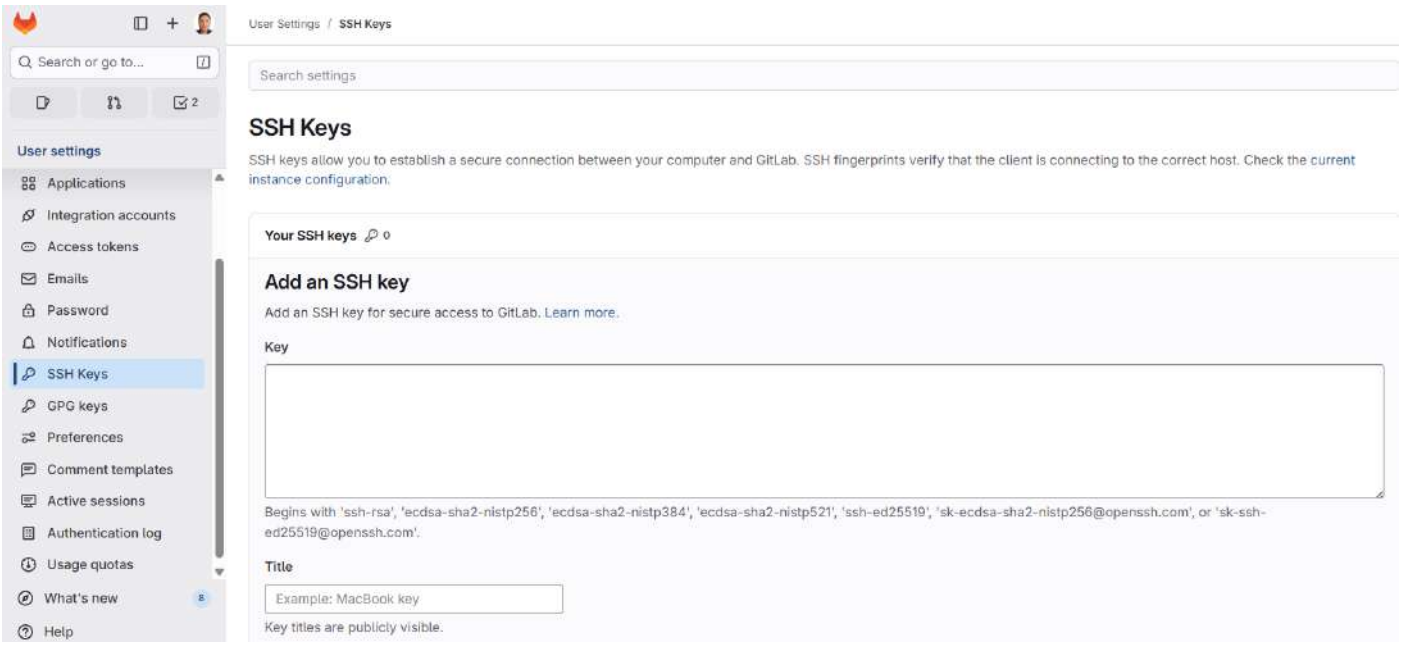
Github

Inicie sesión en Github y abra su configuración personal de usuario a través del menú desplegable superior derecho. Seleccione las claves SSH y GPG y registre su clave:



Gitlab

Inicie sesión en Gitlab y abra la configuración de usuario. En el menú del lado izquierdo, elija la opción Claves SSH y registre el valor de su id_rsa.pubarchivo:



Instalación y configuración por primera vez de GIT

Para instalar diríjase al sitio oficial y prosiga los pasos para su sistema operativo.

CONFIG: Para la configuración por primera vez: En este punto deberás configurar tu identidad y opcionalmente el editor. Para configurar tu identidad

```
$ git config --global user.name "John Doe"

$ git config --global user.email johndoe@example.com
```

Obteniendo un repositorio Git

Para obtener un repositorio existen dos formas: init y clone

INIT: Si estás empezando a seguir un proyecto existente en Git, debes ir al directorio del proyecto y usar el siguiente comando:

```
$ git init
```

Esto crea un subdirectorio nuevo llamado `.git`, el cual contiene todos los archivos necesarios del repositorio

CLONE: Si deseas obtener una copia de un repositorio Git existente — por ejemplo, un proyecto en el que te gustaría contribuir — el comando que necesitas es git clone. Puedes clonar un repositorio con `git clone [url]`. Por ejemplo, si quieres clonar la librería de Git llamada libgit2 puedes hacer algo así:

```
$ git clone https://github.com/libgit2/libgit2
```

Esto crea un directorio llamado `libgit2`, inicializa un directorio `.git` en su interior, descarga toda la información de ese repositorio y saca una copia de trabajo de la última versión.

Ahora que ya tienes configurado tu repositorio podemos continuar con ignorar archivos

Ignorar Archivos

A veces, tendrás algún tipo de archivo que no quieres que Git añada automáticamente o más aun, que ni siquiera quieras que aparezca como no rastreado. Este suele ser el caso de archivos generados automáticamente como trazas o archivos creados por tu sistema de compilación. En estos casos, puedes crear un archivo llamado `.gitignore` que liste patrones a considerar. Este es un ejemplo de un archivo `.gitignore`:

```
$ cat .gitignore

*.[oa]

*~
```

Conceptos fundamentales: commits, branches, merges, pull requests.

- **Commits**: Un commit es una "captura" de los cambios realizados en un proyecto en un momento específico. Es una unidad de cambio que guarda una versión del código. Cada commit incluye un mensaje descriptivo que explica el propósito de los cambios. En Git, los commits crean un historial que permite revertir o inspeccionar versiones anteriores.
- **Branches**: Las branches o "ramas" son versiones paralelas de un proyecto. Permiten trabajar en características nuevas o correcciones sin afectar la rama principal (generalmente llamada main o master). Los desarrolladores pueden hacer commits en estas ramas y luego decidir si integrarlos al proyecto principal.
- **Merges**: El merge es el proceso de combinar cambios de diferentes ramas en una sola. Esto ocurre, por ejemplo, cuando se quiere incorporar una nueva funcionalidad desarrollada en una rama a la rama principal. En Git, los merges pueden ser automáticos si no hay conflictos, o

pueden requerir resolución manual si existen cambios en la misma línea del código en ambas ramas.

- **Pull Requests** (PR): Un pull request es una solicitud para que los cambios realizados en una rama sean revisados e integrados en otra rama (a menudo la rama principal). En plataformas como GitHub, Bitbucket o GitLab, los pull requests permiten revisar, discutir y aprobar cambios antes de integrarlos.

Commit

Para crear nuestro primer commit debemos recordar la estructura de las tres secciones principales de un proyecto de Git: El directorio de Git (Git directory), el directorio de trabajo (working directory), y el área de preparación (staging area).

Vamos a trabajar en el archivo index.html de una página web y queremos cambiar el título de la página.

Modificación:

- Abres `index.html`, cambias el título de `"Mi página web"` a `"Bienvenidos a mi página"`.
- Guardas los cambios en el archivo.

Preparar el cambio (staging):

- Abres la terminal y te aseguras de que estás en el directorio del proyecto.
- Agregas el archivo al área de preparación (staging) para decirle a Git que quieres incluir este cambio en el próximo commit:

```
git add index.html
```

- Este comando indica a Git que registre los cambios de `index.html`.

Realizar el Commit:

```
git commit -m "Actualiza el título de la página principal"
```

Aquí, `-m` permite escribir un mensaje corto y descriptivo de lo que hiciste: "Actualiza el título de la página principal". Este mensaje ayuda a otros (y a ti mismo) a entender qué se cambió.

Branches

Recordemos que los Branch o ramas son versiones paralelas de un proyecto que permiten trabajar en características nuevas o correcciones sin afectar la rama principal (generalmente llamada main o master).

Los comandos para trabajar entre ramas

- **Crear y cambiar a una nueva rama**: git checkout -b nombre-de-la-rama.
- **Ver las ramas**: git branch.
- **Cambiar de rama**: git checkout nombre-de-la-rama.
- **Subir la rama al repositorio remoto**: git push -u origin nombre-de-la-rama.
- **Eliminar una rama localmente**: git branch -d nombre-de-la-rama.
- **Eliminar una rama remota**: git push origin --delete nombre_rama (no afecta a las ramas locales)

Merges

Recordemos que es el proceso de combinar cambios de diferentes ramas en una sola, por ejemplo, de nueva-rama a main o master

Cambia a la rama de destino (a la que quieres incorporar los cambios):

```
git checkout main
```

Realiza el merge desde la rama que deseas incorporar:

```
git merge feature-nueva-funcion
```

Si los cambios en ambas ramas no se superponen, Git hará el merge automáticamente. Sin embargo, si hay modificaciones en las mismas líneas de los mismos archivos, se generarán **conflictos de merge**.

Tipos de Conflictos de Merge

Existen principalmente dos tipos de conflictos:

1. **Conflictos en el Mismo Archivo (conflicto de contenido)**:
   - Este tipo de conflicto ocurre cuando ambas ramas han modificado la misma línea de un archivo.
2. **Conflictos de Eliminación**:
   - Este conflicto sucede cuando una rama ha eliminado un archivo que en la otra rama se ha modificado.

Cómo Solucionar Conflictos de Merge

Cuando Git detecta un conflicto, te lo indicará en la terminal y no completará el merge hasta que los conflictos sean resueltos. Aquí está el proceso para resolverlos:

1. **Ver los archivos en conflicto**:

```
git status
```

   Git mostrará los archivos que tienen conflictos. Normalmente, en los archivos afectados verás marcas como <<<<<<<, =======, y >>>>>>> para identificar los conflictos.

2. **Editar manualmente los archivos en conflicto**:
   - Abre el archivo en un editor de texto.
   - Verás algo como esto:

     <<<<<<< HEAD

     Cambios en la rama actual

     =======

     Cambios en la rama "feature-nueva-funcion"

     >>>>>>> feature-nueva-funcion

- La sección entre <<<<<<< HEAD y ======= muestra los cambios en la rama actual, mientras que la sección entre ======= y >>>>>>> nombre_rama muestra los cambios de la otra rama.

- Edita el archivo y elige qué versión quieres conservar o combina ambas si es necesario.

3. **Marcar el conflicto como resuelto**:

   - Una vez que resuelvas los conflictos, guarda los cambios y agrega el archivo al área de staging:

```
git add nombre_archivo
```

4. **Finalizar el Merge**:

   - Después de resolver todos los conflictos y agregar los archivos, completa el merge:

```
git commit
```

   - Git te pedirá un mensaje para documentar el merge y su resolución.

Comandos Útiles para Manejar Conflictos

- **Descartar los cambios del merge y restaurar la rama a su estado original**:

```
git merge –abort
```

- **Obtener ayuda para revisar y resolver conflictos**:

```
git mergetool
```

  Esto abrirá herramientas de merge si tienes alguna configurada en tu sistema, como KDiff3 o P4Merge.

Resumen para una Clase

"Un merge integra cambios entre ramas en Git. Si ambas ramas han cambiado el mismo archivo en las mismas líneas, se generan **conflictos de merge**. Resolverlos implica abrir el archivo afectado, decidir qué cambios conservar, y luego agregar el archivo al área de staging. Finalmente, se termina el proceso con un commit para registrar el merge y su resolución."

Versionando

Poner un número de versión a tu software se llama **versionado**, y podés hacerlo de varias maneras:

Estándar de numeración

Lo más usado es **SemVer (Versionado Semántico)**:

La especificación de Versionado Semántico ha sido escrita por Tom Preston-Werner, inventor de Gravatar y co-fundador de GitHub.

Dado un número de versión MAYOR.MENOR.PARCHE, se incrementa:

1. La versión MAYOR cuando realizas un cambio incompatible en el API,
2. La versión MENOR cuando añades funcionalidad compatible con versiones anteriores, y
3. La versión PARCHE cuando reparas errores compatibles con versiones anteriores.

Hay disponibles etiquetas para prelanzamiento y metadata de compilación como extensiones al formato MAYOR.MENOR.PARCHE.



Ejemplo



Ejemplo de flujo:

- Versión inicial en desarrollo: 0.1.0
- Versión inicial en producción: 1.0.0
- Agregás funciones nuevas compatibles: 1.1.0
- Corregís un bug: 1.1.1
- Hacés cambios que rompen APIs: 2.0.0

Uso de etiquetas

Alpha:
Versión inicial en desarrollo. No está completa ni estable, puede contener muchos errores y carece de funcionalidades principales. Su propósito es probar ideas, arquitectura y recibir retroalimentación temprana de un grupo reducido de testers o desarrolladores.

Beta:
Versión más avanzada que Alpha. El software ya incluye la mayoría de las funcionalidades previstas, pero aún puede presentar fallos o problemas de rendimiento. Se libera a un público más amplio (beta testers) para detectar errores, validar usabilidad y hacer ajustes antes de la versión final.

Pre-release:
Versión candidata cercana a la final (Release Candidate). El producto está prácticamente terminado y solo se espera detectar errores menores o detalles de compatibilidad antes del lanzamiento estable. Puede considerarse una "vista previa" para usuarios avanzados o testers.

Especificación del Versionado Semántico (SemVer)

Las palabras clave "DEBE", "NO DEBE", "OBLIGATORIO", "DEBERÁ", "NO DEBERÁ", "DEBERÍA", "NO DEBERÍA", "RECOMENDADO", "PUEDE" y "OPCIONAL" en este documento serán interpretadas como se describe en RFC 2119-es.

1. El Software que usa Versionado Semántico DEBE declarar un API público. Este API puede ser declarado en el propio código o debe existir estrictamente en la documentación. Sin importar como sea realizado, este DEBERÍA ser preciso y comprehensivo.

2. Un número de versión normal DEBE tener la forma de X.Y.Z donde X, Y y Z son números enteros no negativos, y NO DEBEN ser precedidos de ceros. X es la versión mayor, Y es la versión menor, y Z es la versión parche. Cada elemento DEBE incrementarse numéricamente. Por ejemplo: 1.9.0 -> 1.10.0 -> 1.11.0

3. Una vez que el paquete versionado ha sido publicado, el contenido de esa versión NO DEBE ser modificado. Cualquier modificación DEBE ser publicada como una nueva versión.

4. Una versión mayor en cero (0.y.z) se considera como desarrollo inicial. Todo PUEDE cambiar en cualquier momento. El API público NO DEBERÍA ser considerado estable.

5. La versión 1.0.0 define el API público. La manera en que cada número de versión es incrementado después de esta publicación dependerá de su API público y cómo cambia.

6.

La versión parche Z (x.y.Z	x > 0) DEBE ser incrementada si solamente se introducen correcciones de errores compatibles con versiones anteriores. Una corrección de error se define como un cambio interno que corrige un comportamiento incorrecto.

7.

La versión menor Y (x.Y.z	x > 0) DEBE ser incrementada si se introduce funcionalidad nueva y compatible con la versión anterior del API público. Ésta DEBE ser incrementada si se introduce cualquier funcionalidad al API público o mejora al código privado. Este PUEDE incluir cambios a nivel de parches. La versión parche DEBE reiniciarse a 0 cuando una versión menor se incrementa.

8.

La versión mayor (X.y.z	X > 0) DEBE ser incrementada solamente si se introducen cambios incompatibles con la versión anterior del API público. Este PUEDE incluir cambios de nivel menor y parches. Versiones parche y menores DEBEN ser reiniciadas a 0 cuando una versión mayor es incrementada.

9. Una versión de prelanzamiento PUEDE ser denotada agregando un guión y una serie de identificadores separados por puntos, inmediatamente seguida de la versión parche. Los identificadores DEBEN ser compuestos sólo de caracteres alfanuméricos ASCII y guión ([0-9A-Za-z-]). Los identificadores NO DEBEN estar vacíos. Identificadores numéricos NO DEBEN ser precedidos de cero. Versiones de prelanzamiento tienen una precedencia inferior que una versión normal. Una versión de prelanzamiento indica que esa versión no es estable y puede no satisfacer los requerimientos de compatibilidad destinados como se denota en la versión normal asociada. Por ejemplo: 1.0.0-alpha, 1.0.0-alpha.1, 1.0.0-0.3.7, 1.0.0-x.7.z.92.

10. Metadatos de compilación PUEDEN ser denotados agregando el signo más y una serie de identificadores separados por puntos, inmediatamente seguido de la versión parche o prelanzamiento. Los identificadores DEBEN ser compuestos de sólo caracteres alfanuméricos ASCII y guión ([0-9A-Za-z-]). Los identificadores NO DEBEN estar vacíos. Identificadores numéricos NO DEBEN ser precedidos de cero. Los metadatos de compilación DEBEN ser ignorados cuando se determina la precedencia de la versión. Por lo tanto, dos versiones que difieren solamente en los metadatos de compilación tienen la misma precedencia. Por ejemplo: 1.0.0-alpha+001, 1.0.0+20130313144700, 1.0.0-beta+exp.sha.5114f85.

11. La precedencia se refiere a cómo las versiones se comparan entre ellas cuando se ordenan.

    1. La precedencia DEBE ser calculada separando los identificadores de la versión en mayor, menor, parche y prelanzamiento (dejando de lado los metadatos de compilación).
    2. La precedencia es determinada por la primera diferencia al comparar cada uno de los identificadores de izquierda a derecha como se indica: mayor, menor y parche son siempre comparadas numéricamente.

       Por ejemplo: 1.0.0 < 2.0.0 < 2.1.0 < 2.1.1.

3. Cuando la versión mayor, menor y parche son iguales, la versión de prelanzamiento tiene menor precedencia que la versión normal.

   Por ejemplo: 1.0.0-alpha < 1.0.0.

4. La precedencia para dos versiones de prelanzamiento con la misma versión mayor, menor y parche DEBE ser determinada comparando cada identificador separado por punto, de izquierda a derecha, hasta que una diferencia sea encontrada como se indica:

   1. identificadores compuestos solamente por números son comparados numéricamente

   2. los identificadores con letras o guiones son comparados léxicamente en orden ASCII.

   3. Identificadores numéricos siempre tienen menor precedencia que identificadores no numéricos.

   4. Un conjunto de campos de prelanzamiento más numeroso tiene mayor precedencia que un conjunto menos numeroso, si todos los identificadores anteriores son iguales.

   Por ejemplo: 1.0.0-alpha < 1.0.0-alpha.1 < 1.0.0-alpha.beta < 1.0.0-beta < 1.0.0-beta.2 < 1.0.0-beta.11 < 1.0.0-rc.1 < 1.0.0.

Selección de versiones en gestores de dependencias

Cuando trabajamos con proyectos de software, necesitamos instalar librerías de terceros. Para evitar problemas de compatibilidad, se usan **esquemas de versionado** (generalmente **SemVer: Semantic Versioning**) y **reglas de selección de versiones**.

Versionado Semántico (SemVer)
Se compone de MAJOR.MINOR.PATCH (ejemplo: 2.5.1).

- **MAJOR**: cambios incompatibles.
- **MINOR**: nuevas funcionalidades compatibles.
- **PATCH**: corrección de errores o parches.

- **Selección de versiones**
  Los gestores permiten indicar qué versiones de una librería se aceptan en el proyecto, usando rangos o símbolos:

  - ^1.2.3 → acepta actualizaciones compatibles en MINOR y PATCH (>=1.2.3 <2.0.0).
  - ~1.2.3 → acepta solo actualizaciones de PATCH (>=1.2.3 <1.3.0).
  - 1.2.* → permite cualquier PATCH dentro de la misma MINOR.
  - >=1.0, <2.0 → rango explícito.
  - * → cualquier versión.

Cada gestor tiene pequeñas diferencias en cómo interpreta estos símbolos, pero la lógica base es la misma.

Cuadro comparativo: selección de versiones

Gestor	Lenguaje/Entorno	Archivo de configuración	Símbolos más usados	Particularidades
Npm	JavaScript / Node.js	package.json	^, ~, *, rangos (>=, <=)	Muy flexible, usa SemVer estricto. Por defecto npm install respeta ^.
Composer	PHP	composer.json	^, ~, *, rangos (>=, <)	Compatible con SemVer, pero en entornos legacy a veces no se cumple estrictamente.
Pip	Python	requirements.txt / pyproject.toml	==, >=, <=, ~=	Menos estricto que npm/composer, muchas librerías antiguas no siguen SemVer.

En resumen:

- **npm** es el más estricto y automatizado con SemVer.
- **Composer** sigue SemVer, pero deja más flexibilidad.
- **pip** es más manual y depende de la calidad del versionado de cada librería.

Ejemplo en los tres gestores

1. npm (Node.js / JavaScript)

Archivo: package.json

```
{
  "dependencies": {
    "mi-lib": "^1.2.3"
  }
}
```

Significa: instalar mi-lib versión >=1.2.3 y <2.0.0.

- Acepta parches (1.2.4, 1.2.5, etc.)
- Acepta nuevas MINOR (1.3.0, 1.4.0, etc.)
- No acepta 2.0.0 porque puede ser incompatible.

2. Composer (PHP)

Archivo: composer.json

```
{
  "require": {
    "mi-lib": "~1.2.3"
  }
}
```

Significa: instalar mi-lib versión >=1.2.3 y <1.3.0.

- Solo se permiten actualizaciones de **PATCH**.
- No acepta 1.3.0 porque podría cambiar el comportamiento.

3. pip (Python)

Archivo: requirements.txt

mi-lib~=1.2.3

📖 Significa: instalar mi-lib versión >=1.2.3 y <1.3.0.

- Igual que Composer, acepta únicamente **PATCH** dentro de la MINOR 1.2.x.
- No acepta 1.3.0 ni 2.0.0.

Comparación rápida del ejemplo

Gestor	Sintaxis usada	Rango permitido en el ejemplo	Nivel de flexibilidad
Npm	"^1.2.3"	>=1.2.3 <2.0.0	Alto (MINOR y PATCH)
Composer	"~1.2.3"	>=1.2.3 <1.3.0	Medio (solo PATCH)
Pip	~=1.2.3	>=1.2.3 <1.3.0	Medio (solo PATCH)

Con esto se ve que:

- **npm** prioriza actualizaciones más amplias (MINOR + PATCH).
- **Composer** y **pip** son más conservadores (solo PATCH).

Con Git

Podés manejar versiones **directamente con Git** usando **tags**:

1. Crear un tag con número de versión:
2. git tag -a v1.0.0 -m "Primera versión estable"
3. Subir el tag al remoto (GitHub, GitLab, etc.):
4. git push origin v1.0.0
5. Ver todas las versiones:
6. git tag

Con esto ya tenés un historial de versiones integrado en tu repo.

Se **puede hacer solo con Git (tags)** y es lo más común. Ahora si querés integrar el número en tu código, changelogs o releases automáticos, ahí conviene sumar otra herramienta.

Automatización (opcional)

- **Herramientas externas** como bump2version, semantic-release o standard-version te ayudan a:
  - Aumentar automáticamente la versión en setup.py, package.json, composer.json, etc.
  - Crear un tag en Git.
  - Generar changelogs automáticos.

Ejemplo con bump2version (Python):

```
bump2version patch   # pasa de 1.0.0 a 1.0.1
bump2version minor   # pasa de 1.0.0 a 1.1.0
bump2version major   # pasa de 1.0.0 a 2.0.0
```

Ejemplo sencillo con Python

1. Crear un tag de versión en Git

En tu repositorio:

```
git tag -a v1.0.0 -m "Primera versión estable"
git push origin v1.0.0
```

Con eso tu repo ya tiene la versión 1.0.0.

2. Obtener la versión desde Git en Python

Podés consultar el tag más reciente desde tu código:

```
import subprocess

def get_version():
    try:
        version = subprocess.check_output(
            ["git", "describe", "--tags"],
            stderr=subprocess.STDOUT
        ).decode().strip()
    except Exception:
        version = "0.0.0"  # valor por defecto si no hay tags
    return version

print("Versión:", get_version())
```

Esto devuelve v1.0.0 si existe el tag en Git.

3. Guardar la versión en un archivo (para evitar depender de Git en producción)

Muchos proyectos crean un archivo version.py:

```
# version.py
__version__ = "1.0.0"
```

Y en tu aplicación:

```
from version import __version__
print("Versión:", __version__)
```

Cuando cambies de versión, solo actualizás ese archivo (o lo actualizás automáticamente con un script que lea el tag de Git).

4. Automatizar con setuptools-scm (opcional)

Si publicás paquetes, podés usar setuptools-scm, que toma automáticamente la versión de tus tags en Git:

```
pip install setuptools-scm
```

En pyproject.toml:

```
[tool.setuptools_scm]
version_scheme = "guess-next-dev"
local_scheme = "node-and-date"
```

Luego, en Python:

```
import importlib.metadata

print(importlib.metadata.version("tu-paquete"))
```

En resumen:

- **Lo más simple** → usar git tag y leerlo con subprocess.
- **Más limpio** → mantener un version.py.
- **Profesional (paquetes PyPI)** → setuptools-scm.

Bibliografía

Versionado Semántico de La especificación de Versionado Semántico ha sido escrita por Tom Preston-Werner, inventor de Gravatar y co-fundador de GitHub, traducido por Italo Baeza Cabrera. https://semver.org/lang/es/

"Version Control with Git" de Jon Loeliger y Matthew McCullough

GIT, website oficial: https://git-scm.com/